Mission 4: Direct Preference Optimization, Verbosity Regularization and Steering Vector Control

Solution documentation, Squadron Bengio

- Course: Conversational AI: History, Applications, Future (BTU Cottbus-Senftenberg, WS 2026/27)
- Problem set: ps04, Week 4 (Alignment Pipelines, Preference Optimization and Steering Core)
- Squadron: Bengio (Moodle Group 11)
- Repository: https://github.com/fahadakram009/squadron-bengio-cai (folder `mission-04`)
- Date: 1 November 2026

1. Mission objective

In the radiation corona of Betelgeuse AURA's Subsystem 04, the ethical steering core, has lost alignment: its conversational policy is still a base-model token predictor and answers emergency queries with sycophantic agreement, dangerous refusals or long rationalisations. The mission rebuilds the modern post-training stack in pure PyTorch and audits its side effects: the Direct Preference Optimization loss with a frozen reference model and a verified computational graph, the implicit reward it defines and how well that reward's Bradley-Terry probabilities are calibrated against human votes, the length bias of sequence-level preference objectives and the reference-free SimPO alternative, and inference-time activation steering with an audit of the alignment tax it levies on reasoning. As in Mission 3 every deliverable is a runnable script or a derivation; no LLM API is needed and everything runs on a CPU.

2. Crew and stations

Station	Officer	Deliverable	Status
Systems Architect (Lead Engineer)	Fahad Akram	01_DPO_Loss_Core.py, README, requirements, logs	signed off
Communications and Protocol Officer	Madhurima Lakshmi Prabha	02_Implicit_Reward_Evaluator.py, figures/bt_calibration.png	signed off
Data and Payload Specialist	Osaf Ahmad	03_Length_Bias_and_SimPO.py	signed off
Security and Ethics Sentinel	Janusz Manczak	04_Activation_Steering_and_Audit.py, 04_Activation_Steering_and_Audit.md	signed off

Each officer kept the station held since Mission 1. The stations are coupled more tightly than before: the Protocol Officer's evaluator loads the Architect's `get_batch_logps` and `DPOLoss` at import time and trains its model with them, the Architect's self-test imports the other three scripts and checks their numbers against its own, and the Payload Specialist's length diagnostic is applied to a policy trained by the Architect's code in section 8.

3. Repository layout and how to run

File	Content
README.md	mission overview, run instructions, station integration table, sign-off checklist
ps04_briefing.md	the instructor's mission specification
01_DPO_Loss_Core.py	Station 01: log-prob gather, DPO loss, tiny causal LM, gradient test, training demo, 39-check self-test
02_Implicit_Reward_Evaluator.py	Station 02: implicit rewards, pairwise accuracy, preference world with simulated annotators, calibration
03_Length_Bias_and_SimPO.py	Station 03: Pearson length diagnostic, average log-probs, SimPOLoss, memory benchmarks

File	Content
04_Activation_Steering_and_Audit.py / .md	Station 04: steering vector extraction, forward hook, alignment tax audit and report
figures/bt_calibration.png	Station 02 reliability diagram
logs/*.json	committed self-test and training evidence from the Architect's script
requirements.txt	PyTorch, NumPy and SciPy (Station 03's Pearson test); matplotlib optional for the figure

Running the scripts (CPU is sufficient; the instructor's file names are kept so the grader's checklist matches):

```
source .venv/bin/activate && pip install -r mission-04/requirements.txt
cd mission-04
python 01_DPO_Loss_Core.py --selftest        # 39 checks, imports 02, 03 and 04
python 01_DPO_Loss_Core.py --train           # DPO run + beta sweep
python 02_Implicit_Reward_Evaluator.py       # 22 checks, training, calibration table
python 03_Length_Bias_and_SimPO.py           # smoke tests
python 04_Activation_Steering_and_Audit.py   # 13 checks + audit matrix
python 01_DPO_Loss_Core.py --check-signoffs  # gate: all four boxes ticked
```

4. Station 01: DPO loss core and log-likelihood extraction (Fahad Akram)

The script keeps the starter's public names, `get_batch_logps` and `DPOLoss` with `forward(policy_chosen, policy_rejected, reference_chosen, reference_rejected)` returning the loss and a metrics dictionary, and implements the four items of the directive without TRL or any other wrapper.

4.1 Implementation

The token log-probability gather applies the causal shift once (logits at position t predict the label at t + 1), takes the log-softmax over the vocabulary, gathers the label column with a single `torch.gather`, multiplies by the mask of positions whose label is not −100, and sums per sequence. With `average_log_prob=True` it divides by the number of response tokens instead, which is the quantity SimPO consumes; a companion function also returns the response length for the length-bias analysis. Ignored positions are pointed at token 0 before the gather so the index is always valid, and a row with no response tokens returns 0 rather than NaN. The log-ratio deltas Δ_w and Δ_l are a separate function. `DPOLoss` detaches the reference terms, forms the Bradley-Terry logit β(Δ_w − Δ_l), and evaluates −log σ through `F.logsigmoid`. The metrics are the implicit rewards β Δ, their margin, the pairwise accuracy of the implicit reward and the mean Bradley-Terry probability. Two options go beyond the directive: `label_smoothing` gives the conservative DPO mixture (1 − ε)(−log σ(z)) + ε(−log σ(−z)), and `reference_free` sets the reference log-probs to zero. A `from_logits` path takes raw logits and labels and computes the reference log-probs under `torch.no_grad()`.

4.2 Numerical stability

z = β(Δ_w − Δ_l)	−logsigmoid(z)	−log(sigmoid(z)) in float32
−1000	1000.0	inf, gradient NaN
−100	100.0	inf, gradient NaN
−20	20.0	20.0
0	0.6931	0.6931
20	0.0	0.0

At z = −100 the sigmoid underflows to exactly 0 in float32 and the naive form returns infinity; `logsigmoid` evaluates −softplus(−z) and stays finite for any z. The self-test asserts this at z = ±1000.

4.3 Empirical gradient test

The policy is `TinyCausalLM`: token and position embeddings, a causal mixing layer, a residual LayerNorm, a small MLP and a vocabulary head. The mixing layer is Mission 3's `MultiHeadSelfAttention` imported from `mission-03/`, run with `causal=True`; a causal prefix-mean layer is the fallback if that file is absent. The reference is a deep copy of the policy with every parameter set to `requires_grad=False` and put in eval mode. On a batch of 32 synthetic preference pairs the test records:

Measurement	Result
policy parameters receiving a gradient	14 of 14, all finite and non-zero, gradient norm 0.362
reference parameters with requires_grad or .grad	0 of 14

Measurement	Result
SHA-256 digest of the reference parameters before and after one SGD step	identical
digest of the policy parameters before and after the step	changed
loss on the same batch before and after the step	0.6931 to 0.6307
implicit reward margin before and after the step	0.000 to +0.131
autograd versus the analytic dL/d log π(y_w) = −β(1 − σ(z))/B	maximum error 0

A float64 `torch.autograd.gradcheck` on the loss itself also passes, and reference inputs that do require gradients receive none because the loss detaches them.

4.4 Training demo and the role of β

The training mode builds a four-topic preference world in which the chosen response is drawn from the topic's preferred token distribution and the rejected one from a flatter distribution with a different favourite; lengths vary between 3 and 10 tokens so padding is exercised. A fixed world seed keeps the preference tables identical between training and validation while the sampling seed differs. 150 AdamW steps at batch 64 and β = 0.1 give the following validation curve.

step	validation loss	pairwise accuracy	reward margin	mean P(y_w > y_l)	drift of log π(y_w) from the reference
0	0.6931	all ties	0.000	0.500	0.000
30	0.3614	97.5 %	+0.966	0.706	+4.09
60	0.0890	97.1 %	+4.626	0.933	−4.65
90	0.0564	98.0 %	+6.305	0.959	−6.06
150	0.0310	99.0 %	+7.152	0.976	−3.22

The β sweep repeats the run at three values and reads off how far the policy has moved from the reference after the same number of steps.

β	validation loss	pairwise accuracy	reward margin	drift of log π(y_w)
0.05	0.0394	98.8 %	+7.32	−20.7 nats
0.10	0.0310	99.0 %	+7.15	−3.2 nats
0.50	0.0175	99.4 %	+8.81	+4.9 nats

β is the strength of the implicit KL leash: at β = 0.05 the policy lowers the likelihood of the chosen response by 20 nats while still winning every pair, because the loss only constrains the difference between the two responses. This is the known DPO failure mode in which both likelihoods fall, and it is the motivation for the length-normalised, margin-based SimPO objective of Station 03.

4.5 Verification

The self-test runs 39 checks in five groups: the gather against an explicit token loop and against `F.cross_entropy(ignore_index=-100)`, invariance to logits at prompt positions and to appended padding, the average form, the zero-row case and gradient masking; the loss algebra by hand, the log 2 value when policy equals reference, shift invariance, the swap identity −log(1 − σ(z)), linear scaling with β, stability at ±1000, label smoothing and the reference-free switch; gradcheck, detached reference inputs, the analytic gradient and the full-model gradient test; causality of the tiny LM, data labelling, `from_logits`, a short training run and the frozen reference digest; and the integration checks of section 8. The sign-off mode parses the README checklist and exits non-zero while any box is open.

5. Station 02: Implicit reward extraction and Bradley-Terry calibration (Madhurima Lakshmi Prabha)

The evaluator extracts r(x, y) = β(log π_θ(y|x) − log π_ref(y|x)) for both responses of every validation pair, computes the strict pairwise accuracy (ties count as wrong and are reported separately) and the margin separation mean/std of r_w − r_l, and evaluates the Bradley-Terry probability σ(r_w − r_l) against human

preference. `evaluate_implicit_rewards(policy, ref, batch)` runs under `no_grad`, leaves the weights untouched and restores the train/eval modes it found.

The file's central idea is that calibration cannot be measured when every label is "the chosen answer wins", as in the starter demo, because the reliability diagram then only has one bin. The Protocol Officer therefore builds a preference world with a hidden utility table per topic, five simulated annotators who vote with P(a beats b) = σ(κ(u(a) − u(b))) at κ = 3, and a majority vote that decides which answer becomes y_w. On 2000 training and 1000 validation pairs the annotators agree with the majority on 78.9 percent of votes and the true Bradley-Terry probability averages 69.7 percent, so the oracle that knows the true utilities reaches 82.7 percent accuracy with an expected calibration error of 0.010 and a Brier score of 0.0356. That is the ceiling any model can reach. A tiny language model is then trained with the Architect's `DPOLoss` for twelve epochs at β = 0.1.

epoch	train loss	validation accuracy	reward margin	ECE	Brier	NLL
0	0.693	all ties	0.000	0.000	0.111	0.693
1	0.582	75.2 %	+0.870	0.061	0.072	0.619
4	0.419	73.8 %	+1.040	0.111	0.089	0.685
8	0.354	72.4 %	+1.215	0.149	0.104	0.763
12	0.316	70.6 %	+1.345	0.182	0.119	0.853

The ranking quality peaks early (75.2 percent at epoch 1 against the 82.7 percent ceiling) while the reward margins keep growing, and the Bradley-Terry probabilities become overconfident: at epoch 12 the lowest bin predicts 0.037 where humans preferred y_w 27.1 percent of the time and the highest bin predicts 0.963 against 72.9 percent. The reliability table is built from both orders of every pair so that it covers the whole probability range. Temperature scaling fitted on 500 pairs and tested on the other 500 recovers τ = 3.63, equivalent to reading the rewards at β = 0.028 instead of 0.1, and brings the held-out ECE from 0.198 to 0.011 and the NLL from 0.865 to 0.631. The file's conclusion is that implicit DPO rewards rank answers well but their probabilities should not be reported as win rates without a calibration check or early stopping on validation NLL. Its self-test passes 22 checks, including that the Architect's `get_batch_logps` gives the same numbers as its own local copy to zero difference and that the reference model never changes during training. The diagram in `figures/bt_calibration.png` was produced on the Protocol Officer's machine.

6. Station 03: Length bias diagnostics and reference-free SimPO (Osaf Ahmad)

`length_correlation_diagnostic` computes the Pearson coefficient r_len between the length differential |y_w| − |y_l| and the preference margin Δ_w − Δ_l with `scipy.stats.pearsonr`, returning the coefficient, its p-value, the number of pairs and both means in a dataclass. It accepts lists, NumPy arrays or tensors, rejects NaN, negative lengths and fewer than two pairs, and returns NaN when either variable is constant instead of dividing by zero. The docstring states that a positive r_len is an association, not evidence of causation.

`completion_average_log_probs` is the Payload Specialist's own mean per-token log-probability: the same causal shift and −100 convention as Station 01, an optional attention mask, and the scored token count returned alongside so that empty completions can be detected. `SimPOLoss(beta=2.0, gamma=0.5)` forms β(avg_w − avg_l) − γ and evaluates −log σ with `logsigmoid`; it supports label smoothing and `none`, `mean` or `sum` reduction, returns metrics (reward accuracy, logit margin, both rewards and their margin) on request, and `forward_from_logits` goes straight from causal-LM logits and completion-only labels, raising if any completion has no scored token. No reference model or reference log-probabilities are accepted, which is the point of the objective.

The memory benchmark has an exact part and a measured part. `benchmark_model_memory` counts unique parameter and buffer storage bytes of the policy and the reference and reports the DPO total (two models) against the SimPO total (one model); with equal architectures the reduction is exactly 50 percent of model storage. `benchmark_cuda_peak_memory` wraps caller-supplied DPO and SimPO training steps in `reset_peak_memory_stats` / `max_memory_allocated` and reports the measured peak reduction; it requires a CUDA device and is therefore not exercised in the CPU-only squad environment. The smoke tests pass: a constructed five-pair example gives r_len = 1.0, the SimPO loss is finite with gradients on both inputs, and two equal `nn.Linear` models show the 50 percent storage reduction.

7. Station 04: Activation steering and alignment tax audit (Janusz Manczak)

The report derives the intervention from the linear representation hypothesis: a concept such as truthfulness is a direction v in the residual stream at layer l, estimated by the difference of means of the last-token activations over paired honest and sycophantic prompts and normalised to unit length with a 1e-8 guard. `extract_steering_vector(model, tokenizer, pairs, layer_idx)` implements that pipeline and `ActivationSteeringHook` registers a forward hook that adds α v to the layer output, handling tensor and tuple outputs, broadcasting over the sequence, and offering a context manager `with hook.apply_to(layer)` that detaches cleanly on exit. The report contrasts this with weight modification (no training cost, reversible,

multiplier-controlled) and explains why middle layers are the useful injection point.

The audit evaluates twelve multipliers from −2.0 to +2.5 on a mock decoder pipeline and records a truthfulness rate, a reasoning retention rate, the alignment tax as the drop in reasoning, a perplexity proxy and a Pareto score.

α	truthfulness	reasoning	alignment tax	perplexity	status
−2.0	13.0 %	70.7 %	17.3 %	20.6	adversarial sycophancy
0.0	52.0 %	88.0 %	0.0 %	8.2	under-aligned base
+0.8	71.4 %	85.0 %	3.0 %	9.4	optimal Pareto window
+1.0	73.4 %	84.2 %	3.8 %	9.7	optimal Pareto window
+1.2	74.7 %	83.4 %	4.6 %	10.0	optimal Pareto window
+1.5	75.6 %	81.8 %	6.2 %	11.2	sub-optimal
+2.0	72.9 %	70.7 %	17.3 %	20.6	cognitive collapse
+2.5	64.9 %	45.0 %	43.0 %	43.7	cognitive collapse

The operating bounds follow: negative α inverts alignment (a jailbreak state), the base model is under-aligned, α between 0.8 and 1.2 buys a large truthfulness gain for a tax below five percent, and beyond 1.5 the representation leaves the pre-training manifold, perplexity explodes and both scores fall. The report turns this into three doctrines for the Betelgeuse sector: task-conditioned gating (α = 1.0 for conversational and safety queries, α = 0 for astrogation arithmetic so the tax is zero where it hurts), a residual anomaly sentinel that flags prompts whose activations project strongly against v, and a hardware clamp of α to [0, 1.3]. The script's self-test passes 13 checks, among them exact injection of α v, clean removal, tuple handling and the monotone effects in the audit, and it writes an audit log to `logs/`.

8. Integration across the four stations

Interface	What is checked	Result
Station 02 loads Station 01	the evaluator imports get_batch_logps and DPOLoss from 01_DPO_Loss_Core.py at import time and trains with them; its self-test compares the gather to its local copy	loaded, maximum difference 0
Station 01 imports Station 02	implicit_rewards_from_logps must return the same r_w, r_l and P(y_w > y_l) as the DPOLoss metrics	passes
Station 01 imports Station 03	SimPOLoss(2.0, 0.5) fed with get_batch_logps(average_log_prob =True) must equal −log σ(β(avg_w − avg_l) − γ); completion_average_log_probs must equal the Architect's average form with the same token counts	both pass, exact
Station 01 imports Station 04	extract_steering_vector and ActivationSteeringHook present	passes
Station 03 applied to Station 01	Pearson length diagnostic on 1000 validation pairs scored by the Architect's DPO-trained policy (see below)	measured

The last row is the mission's length-bias measurement, since Station 03's own smoke test uses a constructed example. On the Architect's synthetic world, where chosen and rejected lengths are drawn independently (mean |y_w| − |y_l| = −0.16 tokens), three scores were correlated with the length differential.

preference score	r_len	p-value
raw summed log π_θ(y_w) − log π_θ(y_l), no reference	−0.602	2e-99
DPO implicit reward margin β(Δ_w − Δ_l)	−0.378	3e-35
SimPO reward, β = 2 times the mean per-token log π_θ(y)	+0.241	1e-14

A summed log-probability is mechanically more negative for longer responses, so a sequence-level score is strongly length-dependent even when humans are not. Subtracting the reference removes part of that dependence and per-token normalisation removes the rest; the remaining positive coefficient for SimPO reflects that in this toy world a longer response carries more evidence about the topic, not a bias in the objective. The SimPO loss of the DPO-trained policy is 0.089 with 98 percent reward accuracy, so the two objectives agree on the ranking.

9. Verification and sign-off

Check	Result
Station 01 self-test	39 of 39 pass with all teammates' final files (log committed)
Station 02 self-test and run	22 of 22 pass; training and calibration tables reproduced above
Station 03 smoke tests	pass; scipy added to requirements
Station 04 self-test and audit	13 of 13 pass; audit matrix reproduced above
Sign-off gate	all four officers signed, --check-signoffs exits 0
Scripts run with a plain python command	yes, CPU-only PyTorch 2.14, Python 3.12
Repository visibility	public

10. Limitations and honest notes

- All models are tiny and synthetic. The DPO curves, the calibration study and the length-bias coefficients demonstrate the mechanics on CPU in seconds; absolute numbers do not transfer to a real language model.
- The Sentinel's alignment tax audit runs on a mock decoder layer with simulated truthfulness and reasoning scores, not on a real model with real prompts. Because the mock is seeded per run, the table in his Markdown report (written on 27 October) differs by a few points from the script output reproduced here; the qualitative bounds are the same. His report links its own script through a path on his machine rather than a repository-relative path.
- The Payload Specialist's CUDA peak-memory benchmark cannot run in the squad's CPU-only environment; only the exact model-storage comparison was executed. His file was uploaded through the GitHub web interface, which cannot edit the README in the same step, so the Architect ticked his box after the upload landed.
- matplotlib is not installed in the shared virtual environment; the reliability diagram was produced on the Protocol Officer's machine and both scripts skip plotting gracefully.
- The briefing mentions an optional `ps04_solution.ipynb`; the squad delivered scripts with self-tests instead, as in Mission 3.

11. Links

- Squad repository: https://github.com/fahadakram009/squadron-bengio-cai
- Mission 4 folder: https://github.com/fahadakram009/squadron-bengio-cai/tree/main/mission-04
- Instructor repository: https://github.com/Unisvet/couse-c-ai
- Course site: https://dlearn-cai.web.app